

ui-ux-pro-max

Comprehensive design guide for web and mobile applications. Contains 67 styles, 96 color palettes, 57 font pairings, 99 UX guidelines, and 25 chart types across 13 technology stacks. Searchable database with priority-based recommendations.

Prerequisites

Check if Python is installed:

```
python3 --version || python --version
```

If Python is not installed, install it based on user's OS:

macOS:

```
brew install python3
```

Ubuntu/Debian:

```
sudo apt update && sudo apt install python3
```

Windows:

```
winget install Python.Python.3.12
```

How to Use This Skill

When user requests UI/UX work (design, build, create, implement, review, fix, improve), follow this workflow:

Step 1: Analyze User Requirements

Extract key information from user request:

- **Product type:** SaaS, e-commerce, portfolio, dashboard, landing page, etc.
- **Style keywords:** minimal, playful, professional, elegant, dark mode, etc.
- **Industry:** healthcare, fintech, gaming, education, etc.
- **Stack:** React, Vue, Next.js, or default to `html-tailwind`

Step 2: Generate Design System (REQUIRED)

Always start with `--design-system` to get comprehensive recommendations with reasoning:

```
python3 skills/ui-ux-pro-max/scripts/search.py "<product_type> <industry>
<keywords>" --design-system [-p "Project Name"]
```

This command:

1. Searches 5 domains in parallel (product, style, color, landing, typography)
2. Applies reasoning rules from `ui-reasoning.csv` to select best matches
3. Returns complete design system: pattern, style, colors, typography, effects
4. Includes anti-patterns to avoid

Example:

```
python3 skills/ui-ux-pro-max/scripts/search.py "beauty spa wellness service" --
design-system -p "Serenity Spa"
```

Step 2b: Persist Design System (Master + Overrides Pattern)

To save the design system for hierarchical retrieval across sessions, add `--persist`:

```
python3 skills/ui-ux-pro-max/scripts/search.py "<query>" --design-system --
persist -p "Project Name"
```

This creates:

- `design-system/MASTER.md` — Global Source of Truth with all design rules
- `design-system/pages/` — Folder for page-specific overrides

With page-specific override:

```
python3 skills/ui-ux-pro-max/scripts/search.py "<query>" --design-system --
persist -p "Project Name" --page "dashboard"
```

This also creates:

- `design-system/pages/dashboard.md` — Page-specific deviations from Master

How hierarchical retrieval works:

1. When building a specific page (e.g., "Checkout"), first check `design-system/pages/checkout.md`
2. If the page file exists, its rules **override** the Master file

3. If not, use `design-system/MASTER.md` exclusively

Step 3: Supplement with Detailed Searches (as needed)

After getting the design system, use domain searches to get additional details:

```
python3 skills/ui-ux-pro-max/scripts/search.py "<keyword>" --domain <domain> [-n <max_results>]
```

When to use detailed searches:

Need	Domain	Example
More style options	<code>style</code>	<code>--domain style "glassmorphism dark"</code>
Chart recommendations	<code>chart</code>	<code>--domain chart "real-time dashboard"</code>
UX best practices	<code>ux</code>	<code>--domain ux "animation accessibility"</code>
Alternative fonts	<code>typography</code>	<code>--domain typography "elegant luxury"</code>
Landing structure	<code>landing</code>	<code>--domain landing "hero social-proof"</code>

Step 4: Stack Guidelines (Default: html-tailwind)

Get implementation-specific best practices. If user doesn't specify a stack, **default to html-tailwind**.

```
python3 skills/ui-ux-pro-max/scripts/search.py "<keyword>" --stack html-tailwind
```

Available stacks: `html-tailwind`, `react`, `nextjs`, `vue`, `svelte`, `swiftui`, `react-native`, `flutter`, `shadcn`, `jetpack-compose`

Search Reference

Available Domains

Domain	Use For	Example Keywords
<code>product</code>	Product type recommendations	SaaS, e-commerce, portfolio, healthcare, beauty, service
<code>style</code>	UI styles, colors, effects	glassmorphism, minimalism, dark mode, brutalism
<code>typography</code>	Font pairings, Google Fonts	elegant, playful, professional, modern
<code>color</code>	Color palettes by product type	saas, ecommerce, healthcare, beauty, fintech, service

Domain	Use For	Example Keywords
landing	Page structure, CTA strategies	hero, hero-centric, testimonial, pricing, social-proof
chart	Chart types, library recommendations	trend, comparison, timeline, funnel, pie
ux	Best practices, anti-patterns	animation, accessibility, z-index, loading
react	React/Next.js performance	waterfall, bundle, suspense, memo, rerender, cache
web	Web interface guidelines	aria, focus, keyboard, semantic, virtualize
prompt	AI prompts, CSS keywords	(style name)

Available Stacks

Stack	Focus
html-tailwind	Tailwind utilities, responsive, a11y (DEFAULT)
react	State, hooks, performance, patterns
nextjs	SSR, routing, images, API routes
vue	Composition API, Pinia, Vue Router
svelte	Runes, stores, SvelteKit
swiftui	Views, State, Navigation, Animation
react-native	Components, Navigation, Lists
flutter	Widgets, State, Layout, Theming
shadcn	shadcn/ui components, theming, forms, patterns
jetpack-compose	Composables, Modifiers, State Hoisting, Recomposition

Example Workflow

User request: "Làm landing page cho dịch vụ chăm sóc da chuyên nghiệp"

Step 1: Analyze Requirements

- Product type: Beauty/Spa service
- Style keywords: elegant, professional, soft
- Industry: Beauty/Wellness
- Stack: html-tailwind (default)

Step 2: Generate Design System (REQUIRED)

```
python3 skills/ui-ux-pro-max/scripts/search.py "beauty spa wellness service  
elegant" --design-system -p "Serenity Spa"
```

Output: Complete design system with pattern, style, colors, typography, effects, and anti-patterns.

Step 3: Supplement with Detailed Searches (as needed)

```
# Get UX guidelines for animation and accessibility  
python3 skills/ui-ux-pro-max/scripts/search.py "animation accessibility" --  
domain ux  
  
# Get alternative typography options if needed  
python3 skills/ui-ux-pro-max/scripts/search.py "elegant luxury serif" --domain  
typography
```

Step 4: Stack Guidelines

```
python3 skills/ui-ux-pro-max/scripts/search.py "layout responsive form" --stack  
html-tailwind
```

Then: Synthesize design system + detailed searches and implement the design.

Output Formats

The `--design-system` flag supports two output formats:

```
# ASCII box (default) - best for terminal display  
python3 skills/ui-ux-pro-max/scripts/search.py "fintech crypto" --design-system  
  
# Markdown - best for documentation  
python3 skills/ui-ux-pro-max/scripts/search.py "fintech crypto" --design-system  
-f markdown
```

Tips for Better Results

1. **Be specific with keywords** - "healthcare SaaS dashboard" > "app"
2. **Search multiple times** - Different keywords reveal different insights
3. **Combine domains** - Style + Typography + Color = Complete design system
4. **Always check UX** - Search "animation", "z-index", "accessibility" for common issues
5. **Use stack flag** - Get implementation-specific best practices
6. **Iterate** - If first search doesn't match, try different keywords

Common Rules for Professional UI

These are frequently overlooked issues that make UI look unprofessional:

Icons & Visual Elements

Rule	Do	Don't
No emoji icons	Use SVG icons (Heroicons, Lucide, Simple Icons)	Use emojis like 🤪 🚀 ⚙️ as UI icons
Stable hover states	Use color/opacity transitions on hover	Use scale transforms that shift layout
Correct brand logos	Research official SVG from Simple Icons	Guess or use incorrect logo paths
Consistent icon sizing	Use fixed viewBox (24x24) with w-6 h-6	Mix different icon sizes randomly

Interaction & Cursor

Rule	Do	Don't
Cursor pointer	Add <code>cursor-pointer</code> to all clickable/hoverable cards	Leave default cursor on interactive elements
Hover feedback	Provide visual feedback (color, shadow, border)	No indication element is interactive
Smooth transitions	Use <code>transition-colors duration-200</code>	Instant state changes or too slow (>500ms)

Light/Dark Mode Contrast

Rule	Do	Don't
Glass card light mode	Use <code>bg-white/80</code> or higher opacity	Use <code>bg-white/10</code> (too transparent)
Text contrast light	Use <code>#0F172A</code> (slate-900) for text	Use <code>#94A3B8</code> (slate-400) for body text
Muted text light	Use <code>#475569</code> (slate-600) minimum	Use gray-400 or lighter
Border visibility	Use <code>border-gray-200</code> in light mode	Use <code>border-white/10</code> (invisible)

Layout & Spacing

Rule	Do	Don't
Floating navbar	Add <code>top-4 left-4 right-4</code> spacing	Stick navbar to <code>top-0 left-0 right-0</code>
Content padding	Account for fixed navbar height	Let content hide behind fixed elements
Consistent max-width	Use same <code>max-w-6xl</code> or <code>max-w-7xl</code>	Mix different container widths

Pre-Delivery Checklist

Before delivering UI code, verify these items:

Visual Quality

- ☐ No emojis used as icons (use SVG instead)
- ☐ All icons from consistent icon set (Heroicons/Lucide)
- ☐ Brand logos are correct (verified from Simple Icons)
- ☐ Hover states don't cause layout shift
- ☐ Use theme colors directly (bg-primary) not var() wrapper

Interaction

- ☐ All clickable elements have `cursor-pointer`
- ☐ Hover states provide clear visual feedback
- ☐ Transitions are smooth (150-300ms)
- ☐ Focus states visible for keyboard navigation

Light/Dark Mode

- ☐ Light mode text has sufficient contrast (4.5:1 minimum)
- ☐ Glass/transparent elements visible in light mode
- ☐ Borders visible in both modes
- ☐ Test both modes before delivery

Layout

- ☐ Floating elements have proper spacing from edges
- ☐ No content hidden behind fixed navbars
- ☐ Responsive at 375px, 768px, 1024px, 1440px
- ☐ No horizontal scroll on mobile

Accessibility

- ☐ All images have alt text
- ☐ Form inputs have labels
- ☐ Color is not the only indicator

- ☐ prefers-reduced-motion respected